BEE 4750 Lab 2: Monte Carlo

Name:

ID:

> **Due Date**
>
> Thursday, 10/02/25, 9:00pm

Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
  Activating project at `c:\lab-2-hollyarcher`
   Installed Libmount_jll         v2.41.2+0
   Installed Xorg_libXfixes_jll      v6.0.2+0
   Installed Xorg_xcb_util_cursor_jll   v0.1.6+0
   Installed Libtiff_jll          v4.7.2+0
   Installed Libuuid_jll          v2.41.2+0
   Installed FillArrays          v1.14.0
   Installed Plots             v1.41.1
Precompiling project...
   4770.1 ms   Libtiff_jll
   6478.5 ms   FillArrays
   4624.6 ms   Libmount_jll
   4665.2 ms   Libuuid_jll
   4695.3 ms   Rmath_jll
```

```
  4757.6 ms    OpenSpecFun_jll
  1424.0 ms    Xorg_libXfixes_jll
  5172.7 ms    QuadGK
  1090.7 ms    Xorg_xcb_util_cursor_jll
  6919.3 ms    ColorSchemes
  1112.2 ms    FillArrays → FillArraysStatisticsExt
  2153.2 ms    FillArrays → FillArraysPDMatsExt
  2242.5 ms    FillArrays → FillArraysSparseArraysExt
  1451.0 ms    Rmath
  1797.2 ms    Fontconfig_jll
  3823.7 ms    SpecialFunctions
  1172.0 ms    ColorVectorSpace → SpecialFunctionsExt
  3526.8 ms    Cairo_jll
  1796.4 ms    HypergeometricFunctions
  4891.4 ms    Qt6Base_jll
  2245.0 ms    HarfBuzz_jll
  2793.9 ms    StatsFuns
  1828.8 ms    libass_jll
  1928.0 ms    Pango_jll
  5737.2 ms    Qt6ShaderTools_jll
 12127.4 ms    PlotUtils
  4200.0 ms    FFMPEG_jll
  3692.9 ms    Qt6Declarative_jll
  2766.9 ms    FFMPEG
  2966.6 ms    GR_jll
  4774.2 ms    PlotThemes
  5685.7 ms    RecipesPipeline
 11805.2 ms    Distributions
  5644.3 ms    GR
  2371.4 ms    Distributions → DistributionsTestExt
 76286.2 ms    Plots
  36 dependencies successfully precompiled in 131 seconds. 155 already precompiled.
```

```julia
using Random # random number generation
using Distributions # probability distributions and interface
using Statistics # basic statistical functions, including mean
using Plots # plotting
```

Overview

In this lab, we will conduct a Monte Carlo experiment and explore how sensitive Monte Carlo estimates are to the underlying probability distribution(s).

You should always start any computing with random numbers by setting a "seed," which controls the sequence of numbers which are generated (since these are not *really* random, just "pseudorandom"). In Julia, we do this with the `Random.seed!()` function.

```
Random.seed!(1)
```

```
TaskLocalRNG()
```

It doesn't matter what seed you set, though different seeds might result in slightly different values. But setting a seed means every time your notebook is run, the answer will be the same.

> **Seeds and Reproducing Solutions**
>
> If you don't re-run your code in the same order or if you re-run the same cell repeatedly, you will not get the same solution. If you're working on a specific problem, you might want to re-use `Random.seed()` near any block of code you want to re-evaluate repeatedly.

Probability Distributions and Julia

Julia provides a common interface for probability distributions with the `Distributions.jl` package. The basic workflow for sampling from a distribution is:

1. Set up the distribution. The specific syntax depends on the distribution and what parameters are required, but the general call is the similar. For a normal distribution or a uniform distribution, the syntax is

   ```
   # you don't have to name this "normal_distribution"
   #   is the mean and   is the standard deviation
   normal_distribution = Normal( ,  )
   # a is the upper bound and b is the lower bound; these can be set to +Inf or -Inf for ar
   uniform_distribution = Uniform(a, b)
   ```

   There are lots of both univariate and multivariate distributions, as well as the ability to create your own, but we won't do anything too exotic here.

2. Draw samples. This uses the `rand()` command (which, when used without a distribution, just samples uniformly from the interval $[0, 1]$.) For example, to sample from our normal distribution above:

   ```
   # draw n samples
   rand(normal_distribution, n)
   ```

Putting this together, let's say that we wanted to simulate 100 six-sided dice rolls. We could use a Discrete Uniform distribution.

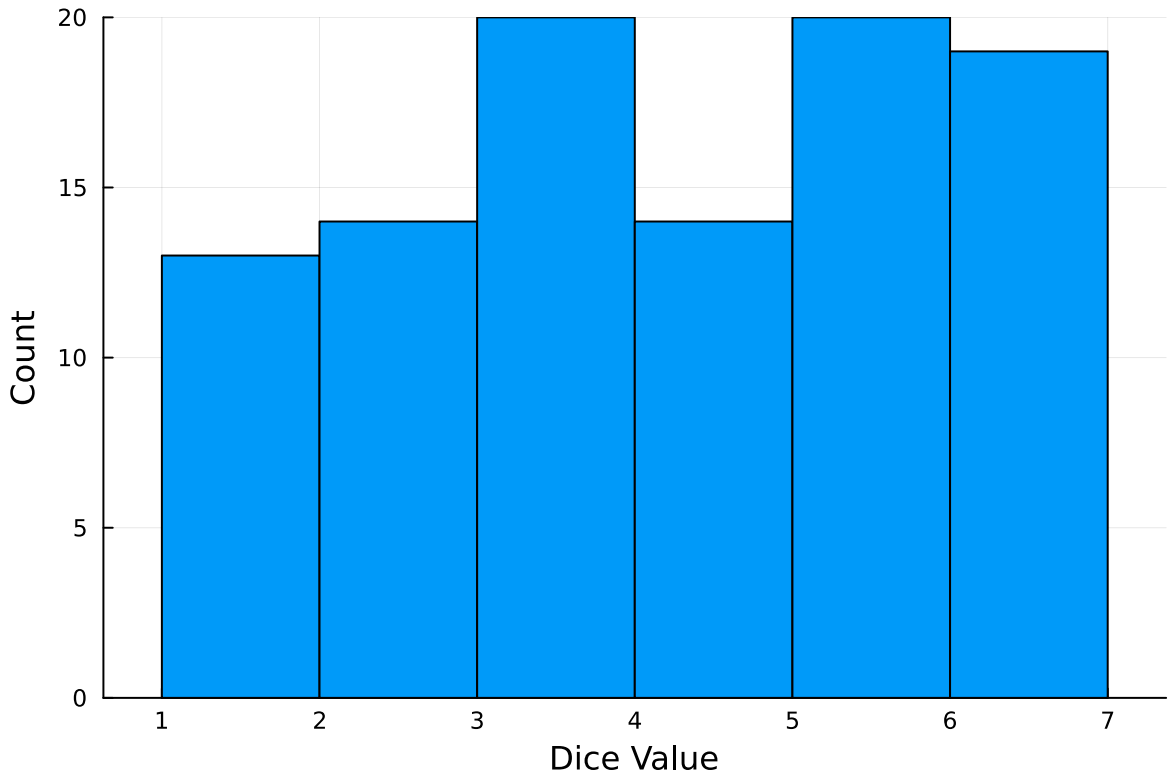
```
dice_dist = DiscreteUniform(1, 6) # can generate any integer between 1 and 6
dice_rolls = rand(dice_dist, 100) # simulate rolls
```

```
100-element Vector{Int64}:
 1
 3
 5
 4
 6
 2
 5
 5
 5
 2

 3
 6
 5
 5
 6
 3
 6
 6
 6
```

And then we can plot a histogram of these rolls:

```
histogram(dice_rolls, legend=:false, bins=6)
ylabel!("Count")
xlabel!("Dice Value")
```



Instructions

Remember to:

- Evaluate all of your code cells, in order (using a `Run All` command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

Problem

A common engineering problem is to quantify flood risk, which is typically computed by propagating a flood hazard distribution through a *depth-damage function* relating flood depths to economic damages. A reasonable depth-damage function for a house without a basement is a bounded logistic function,

$$d(h) = \mathbb{1}_{h>0} \frac{L}{1 + \exp(-k(h - h_0))},$$

where d is the damage as a percent of total structure value, h is the water depth (relative to the house's ground floor elevation) in m, $\mathbb{1}_{x>0}$ is the indicator function, L is the maximum loss in USD, k is the slope of the depth-damage relationship, and h_0 is the inflection point. We'll assume $L = \$200,000$, $k = 0.8$, and $h_0 = 3$.

For this problem, suppose that we have two different probability distributions characterizing annual maxima flood depths:

1. $h \sim LogNormal(1.2, 0.3)$;
2. $h \sim GeneralizedExtremeValue(3, 0.9, -0.15)$.

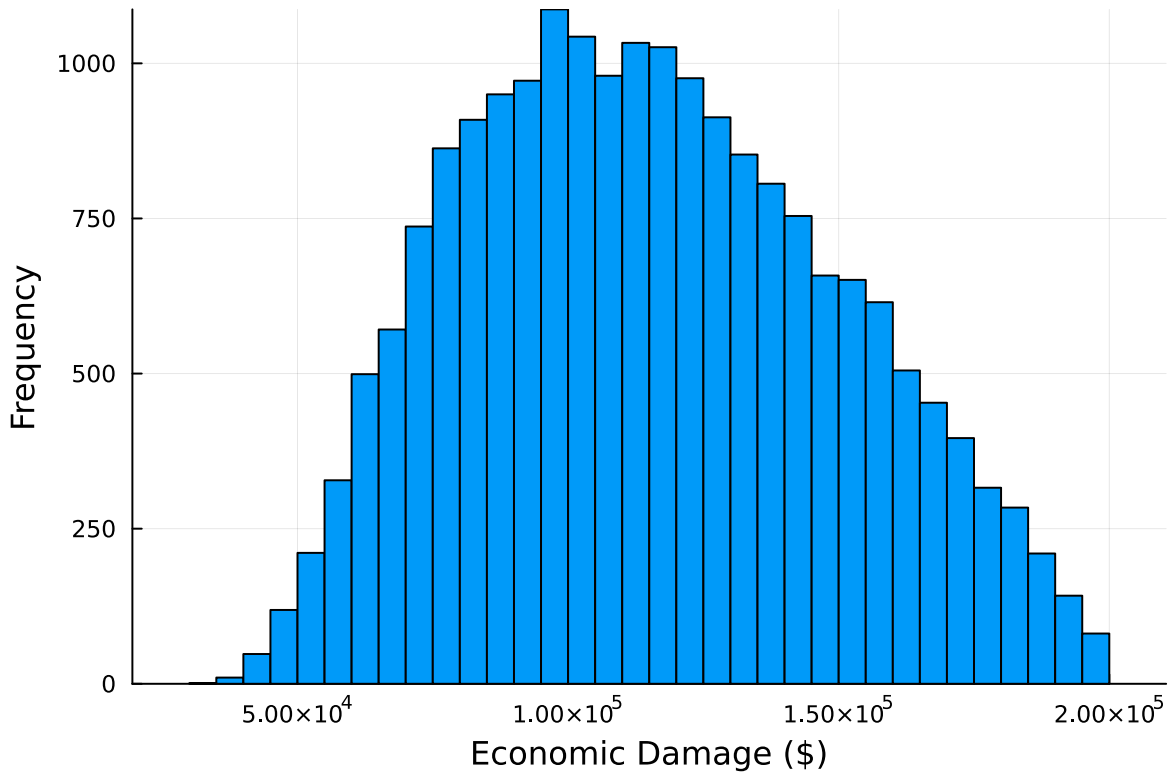
Plot histograms of samples from these distributions and conduct a Monte Carlo experiment for annual maximum flood damages using each. Answer the following questions:

- What are the Monte Carlo estimates of the expected value and the 99% quantile for the annual maximum damage the structure would suffer for each of these flood hazard distributions?
- How did you decide on your sample size?
- Why do you think the estimates differed or did not differ (you can also plot the depth-damage function to compare with the samples)?
- How might you decide (based on getting additional information, if needed) which distribution is "better"?

```
# Distribution 1
depth = LogNormal(1.2, 0.3)
h = rand(depth, 20000)
L = 200000
k = 0.8
h0 = 3
d_h = []
for i in (1:length(h))
    dh = L/(1+exp(-k*(h[i]-h0)))
    push!(d_h, dh)
end
exp_val = mean(d_h)
quantile99 = quantile(d_h, 0.99)

H = histogram(d_h, legend=:false, bins=50, xlabel= "Economic Damage (\$)", ylabel = "Frequen
println("Mean damage: $exp_val, 99% quantile: $quantile99")
display(H)
```

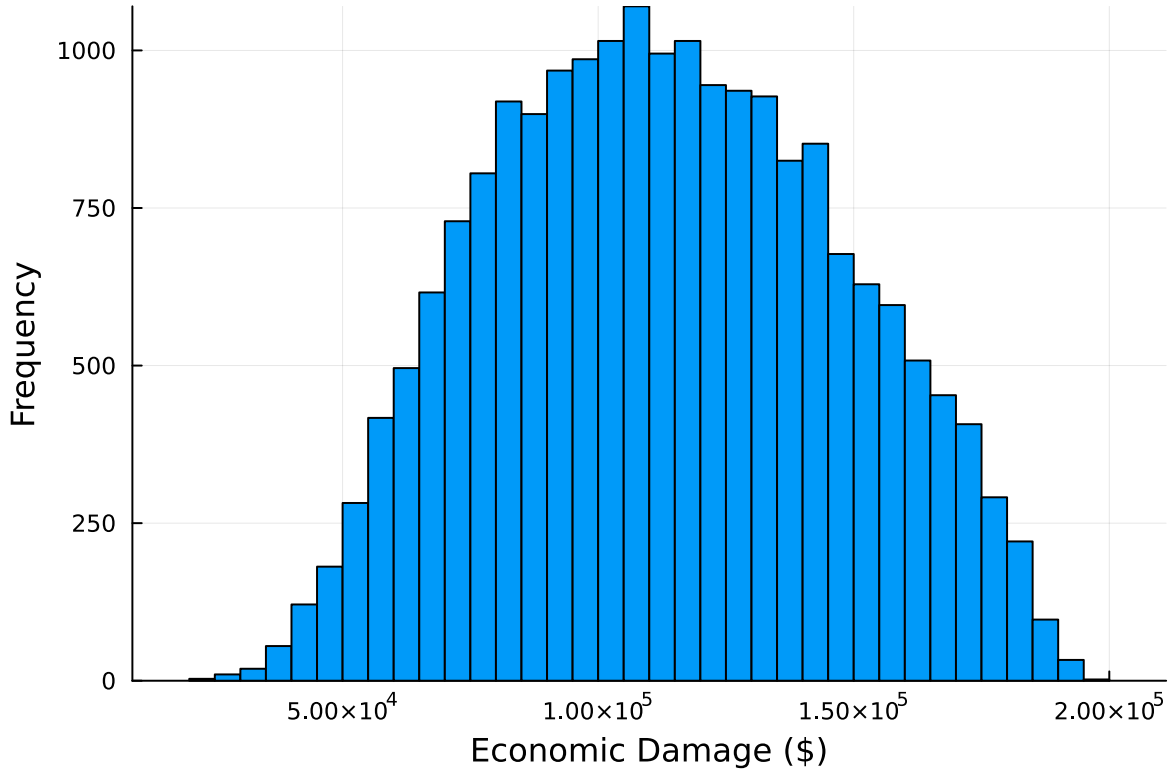
```
Mean damage: 115440.15383231988, 99% quantile: 190621.40089868713
```




```julia
# Distribution 2
depth = GeneralizedExtremeValue(3, 0.9, -0.15)
h = rand(depth, 20000)
L = 200000
k = 0.8
h0 = 3
d_h = []
for i in (1:length(h))
    if h[i]<=0
        indc = 0
    else
        indc = 1
    end
    dh = indc*L/(1+exp(-k*(h[i]-h0)))
    push!(d_h, dh)
end

exp_val = mean(d_h)
quantile99 = quantile(d_h, 0.99)
```

```
H = histogram(d_h, legend=:false, bins=50, xlabel= "Economic Damage (\$)", ylabel = "Frequen
println("Mean damage: $exp_val, 99% quantile: $quantile99")
display(H)
```




```
Mean damage: 113101.78176338784, 99% quantile: 183122.2750286669
```

I decided on my sample size and bin number through trial and error, using the visuals from run to run. I started at 100 runs with 20 bins to just test my code, and after I had it ironed out, I noticed that the plots were changing pretty drastically between each run. I upped it to 1000 and the shape was a little more consistent, and I finally upped it to 20,000 to find a pretty consistent shape run to run, which I was happy with. As for the bins, I continued to increase this number until the plot looked smooth and the shape was more defined rather than jagged. I landed on 50 bins, although anything above that would also be telling for the shape. I didn't want to go too high because it is still a histogram, and I didn't want it to get too granular.

References

Put any consulted sources here, including classmates you worked with/who helped you.